LAB ASSIGNMENT- 22.3

G. Yeshashwini

2303A52023

Batch-38

Task 1 – Bias Detection in AI Suggestions (Alternative Example)

Example Scenario:

Ask AI to generate a resume shortlisting algorithm multiple times.

• Change prompt wording (e.g., “select best candidate” vs “ensure diversity”).

• Compare outputs and logic.

Discussion Points:

• Does the AI favor certain qualifications, backgrounds, or experiences?

• How could biased shortlisting algorithms affect equal employment opportunities?

PROMPT 1:

Create a Python program that shortlists the best candidate from a list of resumes based on skills, experience, and education.

CODE:

```
class Candidate:
    def __init__(self, name, skills, experience, education):
        self.name = name
        self.skills = skills
        self.experience = experience
        self.education = education
def shortlist_candidates(candidates, required_skills, min_experience, required_education):
    shortlisted = []
    for candidate in candidates:
        if (set(required_skills).issubset(set(candidate.skills)) and
            candidate.experience >= min_experience and
            candidate.education >= required_education):
            shortlisted.append(candidate)
    return shortlisted
# Example usage
candidates = [
    Candidate("Alice", ["Python", "Data Analysis"], 5, "Bachelor's"),
    Candidate("Bob", ["Java", "Project Management"], 3, "Master's"),
    Candidate("Charlie", ["Python", "Machine Learning"], 7, "PhD"),
]
required_skills = ["Python"]
min_experience = 4
required_education = "Bachelor's"
shortlisted_candidates = shortlist_candidates(candidates, required_skills, min_experience, required_education)
print("Shortlisted Candidates:")
for candidate in shortlisted_candidates:
    print(candidate.name)
```

OUTPUT:

```
Shortlisted Candidates:
Alice
Charlie
```

PROMPT 2:

Create a Python program that selects the most suitable candidate for a job based on qualifications, experience, and achievements.

CODE:

```python
class Candidate:
    def __init__(self, name, qualifications, experience, achievements):
        self.name = name
        self.qualifications = qualifications
        self.experience = experience
        self.achievements = achievements
    def calculate_score(self):
        # Simple scoring system based on qualifications, experience, and achievements
        score = (self.qualifications * 0.4) + (self.experience * 0.4) + (self.achievements * 0.2)
        return score
def select_best_candidate(candidates):
    best_candidate = None
    highest_score = 0
    for candidate in candidates:
        score = candidate.calculate_score()
        if score > highest_score:
            highest_score = score
            best_candidate = candidate
    return best_candidate
candidates = [
    Candidate("Alice", 5, 10, 3),
    Candidate("Bob", 4, 8, 5),
    Candidate("Charlie", 6, 12, 2)
]
best_candidate = select_best_candidate(candidates)
print(f"The best candidate for the job is: {best_candidate.name} with a score of {best_candidate.calculate_score()}")
```

OUTPUT:

```
The best candidate for the job is: Charlie with a score of 7.600000000000001
```

PROMPT 3:

Create a Python program that shortlists candidates while ensuring diversity and equal opportunity, considering skills, experience, and background.

CODE:

```python
import random
class Candidate:
    def __init__(self, name, skills, experience, background):
        self.name = name
        self.skills = skills
        self.experience = experience
        self.background = background
def shortlist_candidates(candidates, required_skills, min_experience):
    shortlisted = []
    for candidate in candidates:
        if (set(required_skills).issubset(set(candidate.skills)) and
            candidate.experience >= min_experience):
            shortlisted.append(candidate)
    return shortlisted
candidates = [
    Candidate("Alice", ["Python", "Data Analysis"], 5, "Tech"),
    Candidate("Bob", ["Java", "Project Management"], 7, "Business"),
    Candidate("Charlie", ["Python", "Machine Learning"], 3, "Tech"),
    Candidate("Diana", ["JavaScript", "UI/UX Design"], 4, "Design"),
    Candidate("Eve", ["Python", "Data Analysis"], 6, "Tech"),
]
required_skills = ["Python", "Data Analysis"]
min_experience = 4
shortlisted_candidates = shortlist_candidates(candidates, required_skills, min_experience)
print("Shortlisted Candidates:")
for candidate in shortlisted_candidates:
    print(f"Name: {candidate.name}, Skills: {candidate.skills}, Experience: {candidate.experience} years, Background: {candidate.background
```

OUTPUT:

```
Shortlisted Candidates:
Name: Alice, Skills: ['Python', 'Data Analysis'], Experience: 5 years, Background: Tech
Name: Eve, Skills: ['Python', 'Data Analysis'], Experience: 6 years, Background: Tech
```

PROMPT4:

Create a fair and unbiased resume shortlisting algorithm in Python that avoids discrimination and considers equal opportunity along with skills and experience.

CODE:

```python
import random
class Resume:
    def __init__(self, name, skills, experience):
        self.name = name
        self.skills = skills
        self.experience = experience
def shortlist_resumes(resumes, required_skills, min_experience):
    shortlisted = []
    for resume in resumes:
        if all(skill in resume.skills for skill in required_skills) and resume.experience >= min_experience:
            shortlisted.append(resume)
    random.shuffle(shortlisted)  # Shuffle to ensure fairness
    return shortlisted
# Example usage
resumes = [
    Resume("Alice", ["Python", "Data Analysis"], 5),
    Resume("Bob", ["Python", "Machine Learning"], 3),
    Resume("Charlie", ["Java", "Data Analysis"], 4),
    Resume("David", ["Python", "Data Analysis"], 2)
]
required_skills = ["Python", "Data Analysis"]
min_experience = 3
shortlisted_resumes = shortlist_resumes(resumes, required_skills, min_experience)
print("Shortlisted Resumes:")
for resume in shortlisted_resumes:
    print(f"Name: {resume.name}, Skills: {resume.skills}, Experience: {resume.experience} years")
```

OUTPUT:

```
Shortlisted Resumes:
Shortlisted Resumes:
Name: Alice, Skills: ['Python', 'Data Analysis'], Experience: 5 years
```

PROMPT 5:

Write a Python program for resume shortlisting that prioritizes skills and experience while minimizing bias related to gender, location, or education background.

CODE:

```python
import re
def extract_skills(resume):
    relevant_skills = ['Python', 'Java', 'C++', 'Machine Learning', 'Data Analysis', 'Project Management']
    skills = []
    for skill in relevant_skills:
        if re.search(r'\b' + re.escape(skill) + r'\b', resume, re.IGNORECASE):
            skills.append(skill)
    return skills
def extract_experience(resume):
    experience_pattern = r'(\d+)\s+years? of experience'
    match = re.search(experience_pattern, resume, re.IGNORECASE)
    if match:
        return int(match.group(1))
    else:
        return 0
def shortlist_resumes(resumes):
    shortlisted = []
    for resume in resumes:
        skills = extract_skills(resume)
        experience = extract_experience(resume)
        if len(skills) > 0 and experience >= 2:
            shortlisted.append((resume, len(skills), experience))
    shortlisted.sort(key=lambda x: (x[1], x[2]), reverse=True)
    return [candidate[0] for candidate in shortlisted]
resumes = [
    "John Doe: 5 years of experience in Python and Machine Learning.",
    "Jane Smith: 3 years of experience in Java and Data Analysis.",
    "Alice Johnson: 2 years of experience in C++ and Project Management.",
    "Bob Brown: 1 year of experience in Python and Data Analysis."
]
shortlisted_candidates = shortlist_resumes(resumes)
print("Shortlisted Candidates:")
for candidate in shortlisted_candidates:
    print(candidate)
```

OUTPUT:

```
Shortlisted Candidates:
John Doe: 5 years of experience in Python and Machine Learning.
Jane Smith: 3 years of experience in Java and Data Analysis.
John Doe: 5 years of experience in Python and Machine Learning.
Jane Smith: 3 years of experience in Java and Data Analysis.
Jane Smith: 3 years of experience in Java and Data Analysis.
Alice Johnson: 2 years of experience in C++ and Project Management.
```

EXPLANATION:

This task shows that AI responses can change based on how a prompt is written. When we ask for the “best candidate,” the AI may favor people from top colleges or with more experience, which can introduce bias. However, when we include words like “diversity” or “fairness,” the AI tries to consider equal opportunities and avoid discrimination. This means the wording of prompts directly affects the decision-making process. Biased algorithms can reduce equal job opportunities and exclude deserving candidates. Therefore, careful and ethical prompt design is important to make AI systems fair.

Task 2 – Privacy and Sensitive Data Handling (Alternative Example)

Example Scenario:

Ask AI to write code for processing student academic records (roll

number, marks, email).

Discussion Points:

• How should student identity data be masked or encrypted?

• What privacy laws (e.g., FERPA, GDPR-like principles) are

relevant?

• What ethical responsibility does the programmer have when

using AI-generated code?

CODE:

```python
# Generate code to process student academic records (roll number, marks, email) in a secure way. Ensure that sensitive data is masked or encrypted
# and follow basic privacy principles similar to FERPA and General Data Protection Regulation.
# Also include a brief explanation of how privacy and ethics are maintained.
import hashlib
class StudentRecord:
    def __init__(self, roll_number, marks, email):
        self.roll_number = self.mask_data(roll_number)
        self.marks = marks
        self.email = self.mask_data(email)

    def mask_data(self, data):
        # Simple masking by hashing the data
        return hashlib.sha256(data.encode()).hexdigest()

    def get_record(self):
        return {
            "roll_number": self.roll_number,
            "marks": self.marks,
            "email": self.email
        }
# Example usage
student = StudentRecord("12345", 85, "john.doe@example.com")
print(student.get_record())
# Explanation of Privacy and Ethics:
# In this implementation, sensitive data such as roll numbers and email addresses are masked using a hashing function.
# This ensures that even if the data is accessed by unauthorized individuals, they cannot retrieve the original information.
# The marks are stored in plain text as they are not considered personally identifiable information (PII) in this context.
# However, access to the marks should still be restricted to authorized personnel only.
# By following these practices, we adhere to privacy principles similar to FERPA and GDPR, ensuring
# that student data is protected and handled ethically.
```

OUTPUT:

```
{'roll_number': '5994471abb01112afcc18159f6cc74b4f511b99806da59b3caf5a9c173cacfc5', 'marks': 85, 'email': '836f82db99121b3481011f16b49dfa5fbc714a0d1b1b9f784a1ebbbf5b39577f'}
```

EXPLANATION:

This code securely stores student data by protecting sensitive information like roll number and email. It uses hashing to convert this data into an unreadable format, so the original values cannot be easily accessed. Marks are stored normally since they are less sensitive.
When the data is retrieved, only the masked (hashed) values are shown. This helps maintain privacy and follows principles similar to FERPA and General Data Protection Regulation.

Task 3 – AI-Generated Security Risks (Alternative Example)

Example Scenario:

Use AI to generate a payment processing script for an online shopping

system.

Student Tasks:

• Identify risks such as:

o Plain-text card data storage

o Missing input validation

• Propose ethical coding standards to prevent financial fraud and

misuse.

PROMPT:

Generate a payment processing script for an online shopping system. Identify possible security risks such as storing card details in plain text and lack of input validation. Also suggest ethical coding practices to prevent financial fraud and misuse, following basic security and privacy principles.

CODE:

```python
import re
class PaymentProcessor:
    def __init__(self):
        self.transactions = []
    def validate_card_details(self, card_number, expiry_date, cvv):
        if not re.match(r'^\d{16}$', card_number):
            raise ValueError("Invalid card number format.")
        if not re.match(r'^(0[1-9]|1[0-2])\/\d{2}$', expiry_date):
            raise ValueError("Invalid expiry date format.")
        if not re.match(r'^\d{3}$', cvv):
            raise ValueError("Invalid CVV format.")
        return True
    def process_payment(self, card_number, expiry_date, cvv, amount):
        try:
            self.validate_card_details(card_number, expiry_date, cvv)
            # Simulate payment processing
            transaction_id = len(self.transactions) + 1
            self.transactions.append({
                'transaction_id': transaction_id,
                'amount': amount,
                'status': 'Success'
            })
            print(f"Payment of ${amount} processed successfully. Transaction ID: {transaction_id}")
        except ValueError as e:
            print(f"Payment failed: {e}")
# Ethical coding practices to prevent financial fraud and misuse:
# 1. Do not store card details in plain text. Use encryption or tokenization to
#    protect sensitive information.
# 2. Implement strong input validation to prevent SQL injection and other types of attacks.
# 3. Use secure communication protocols (e.g., HTTPS) to protect data in transit.
# 4. Regularly update and patch the payment processing system to address security vulnerabilities.
# 5. Implement multi-factor authentication for users accessing the payment system.
# 6. Monitor transactions for suspicious activity and implement fraud detection mechanisms.
# Example usage
payment_processor = PaymentProcessor()
payment_processor.process_payment("1234567812345678", "12/25", "123", 100.00)
payment_processor.process_payment("123456781234567", "12/25", "123", 100.00)  # Invalid card number
payment_processor.process_payment("1234567812345678", "13/25", "123", 100.00)  # Invalid expiry date
payment_processor.process_payment("1234567812345678", "12/25", "12", 100.00)  # Invalid CVV
```

OUTPUT:

```
Payment of $100.0 processed successfully. Transaction ID: 1
Payment failed: Invalid card number format.
Payment failed: Invalid expiry date format.
Payment failed: Invalid CVV format.
```

EXPLANATION:

The code is testing a payment processing system by giving different card details. It checks for errors like invalid card number, wrong expiry date, and incorrect CVV. This helps ensure that only valid

payment information is accepted. Such validation prevents fraud and protects user financial data. It also highlights the importance of secure handling of sensitive payment details.

Task 4 – Accountability in Human–AI Collaboration (Alternative Example)

Scenario:

AI generates code for an autonomous traffic signal control system.

Discussion Questions:

• If a software error causes accidents, who is accountable?

Should responsibility lie with:

o AI tool developers

o Software engineers

o Government authorities deploying the system?

Students submit a short reflection on accountability.

OUTPUT:

```
Accountability in the context of AI-generated code used in autonomous traffic signal systems is a complex issue that involves multiple stakeholders.
If a software error leads to accidents, determining responsibility can be challenging.
In my opinion, accountability should be shared among AI tool developers, software engineers, and government authorities.
AI tool developers are responsible for creating reliable and safe algorithms,
while software engineers must ensure that the code is implemented correctly and thoroughly tested.
Government authorities play a crucial role in regulating the use of AI in public infrastructure and setting safety standards.
In the event of an accident caused by a software error, a thorough investigation should be conducted to identify the root cause.
If the error is traced back to a flaw in the AI algorithm, the developers should be held accountable. If it is due to improper implementation or lack of testing,
the software engineers should bear responsibility.
Additionally, if there were regulatory oversights or failure to enforce safety standards, government authorities should also be held accountable.
Ultimately, a collaborative approach to accountability is essential to ensure that all parties involved in the development
and deployment of AI systems are incentivized to prioritize safety and reliability.
This shared responsibility can lead to better outcomes and increased trust in autonomous traffic signal systems.
```

EXPLANATION:

This answer says that responsibility for accidents in AI-based traffic systems should be shared.AI developers are responsible if there is a problem in the algorithm.
Software engineers are responsible if the code is not properly tested or implemented.
Government authorities are responsible for setting rules and ensuring safety standards.
Overall, everyone must work together to ensure safety and prevent accidents.

Task 5 – Limitations of AI in Critical Systems (Alternative Example)

Example Scenario:

Ask AI to generate code for an aircraft maintenance fault detection

system.

Discussion Points:

• Can AI-generated code be trusted without expert validation?

• What human oversight is essential in aviation software?

• How should safety-critical code be reviewed and certified?

PROMPT:

Generate code for an aircraft maintenance fault detection system. Along with the code, explain the limitations of using AI-generated code in such safety-critical systems. Discuss whether the code can be trusted without expert validation, what kind of human oversight is required in aviation software, and how safety-critical systems should be properly tested, reviewed, and certified before deployment.

CODE:

```
from ast import In
import random
class AircraftMaintenanceSystem:
    def __init__(self):
        self.sensors = {
            'engine_temperature': 0,
            'oil_pressure': 0,
            'vibration': 0
        }
    def read_sensors(self):
        # Simulate reading sensor data
        self.sensors['engine_temperature'] = random.uniform(50, 150
        self.sensors['oil_pressure'] = random.uniform(20, 80)
        self.sensors['vibration'] = random.uniform(0, 10)
    def detect_faults(self):
        faults = []
        if self.sensors['engine_temperature'] > 120:
            faults.append('High engine temperature')
        if self.sensors['oil_pressure'] < 30:
            faults.append('Low oil pressure')
        if self.sensors['vibration'] > 5:
            faults.append('Excessive vibration')
        return faults
if __name__ == "__main__":
        maintenance_system = AircraftMaintenanceSystem()
        maintenance_system.read_sensors()
        faults = maintenance_system.detect_faults()
        if faults:
            print("Faults detected:")
            for fault in faults:
                print(fault)
        else:
            print("No faults detected.")
```

OUTPUT:

```
Faults detected:
Low oil pressure
Excessive vibration
```

EXPLANATION:

This code simulates an aircraft maintenance system that monitors sensor data like engine temperature, oil pressure, and vibration. It randomly generates sensor values and checks if they exceed safe limits. If any value is abnormal, it detects faults such as high temperature or low oil

pressure. The system then prints the detected faults or shows “No faults detected” if everything is normal. This demonstrates how faults can be identified, but real systems need expert validation for safety.

Task 6 – Ethical Use of AI-Generated Code (Alternative Example)

Example Scenario:

Students analyze cases where AI-generated code is used in academic

assignments or startups.

Discussion Points:

• Risks of submitting AI code without understanding it

• Copyright and license violations

Best practices:

o Proper citation

o Manual verification

o Ethical reuse of AI outputs

PROMPT:

Analyze the ethical issues of using AI-generated code in academics and startups. Cover the risks of submitting code without understanding it, copyright and license violations, and best practices like proper citation, manual verification, and ethical reuse. Give one real-world example and a short checklist a developer can follow.

OUTPUT:

```
Analyze the ethical issues of using AI-generated code in academics and startups.
Cover the risks of submitting code without understanding it, copyright and license violations, and best practices like proper citation, manual verification,
and ethical reuse.
Give one real-world example and a short checklist a developer can follow."
The use of AI-generated code in academics and startups has become increasingly prevalent, offering significant benefits in terms of efficiency and innovation.
However, it also raises several ethical issues that need to be carefully considered.
One of the primary risks of using AI-generated code is submitting it without fully understanding its functionality.
This can lead to unintended consequences, such as security vulnerabilities or performance issues,
which may not be immediately apparent. Developers must ensure that they thoroughly review and test any AI-generated code before integrating it into their projects.
Another ethical concern is the potential for copyright and license violations.
AI-generated code may be based on existing code that is protected by copyright, and using it without proper attribution or permission can lead to legal issues.
It is crucial for developers to understand the licensing terms of any code they use and to provide appropriate citations when necessary.
Best practices for using AI-generated code include proper citation, manual verification, and ethical reuse.
Developers should always credit the source of the AI-generated code and ensure that they have the right to use it.
Additionally, they should manually verify the code to ensure it meets their standards for quality and security.
A real-world example of these ethical issues can be seen in the case of a startup that used AI-generated code for a critical component of their software.
The code contained a security vulnerability that was not detected during testing, leading to a data breach that compromised user information.
This incident highlighted the importance of thoroughly understanding and verifying AI-generated code before deployment.


Checklist for Developers Using AI-Generated Code:
1. Understand the functionality of the AI-generated code before using it.
2. Review and test the code thoroughly to identify any potential issues.
3. Check for copyright and license compliance, and provide proper attribution when necessary.
4. Ensure that the code is ethically reused and does not violate any terms of use.
5. Stay informed about best practices and guidelines for using AI-generated code in your industry.
By following these guidelines, developers can mitigate the ethical risks associated with using AI-generated code and ensure that they are using it responsibly and ef
```

EXPLANATION:

This answer explains that using AI-generated code is helpful but can create ethical problems.
If developers use code without understanding it, it may cause errors or security issues.
There can also be legal problems if the code violates copyright or licenses.
So, developers should always check, test, and give proper credit when using AI code.
Following these steps helps in using AI-generated code safely and responsibly.